AI ENABLE ROAD EXTRACTION - CHENNAI

INTRODUCTION

Problem Statement

Traditional road mapping methods rely on manual digitization, which is time-consuming, labor-intensive, and prone to human error.

Technological Advancement

Recent developments in Artificial Intelligence and Deep Learning enable automatic extraction of road features from high-resolution satellite imagery.

Approach Used

In this project, a U-Net model implemented using PyTorch is used to detect and segment road pixels from satellite images.

Objective

To develop an automated system for road extraction from satellite imagery using AI techniques.

Study Area

The methodology is applied to satellite imagery of Chennai.

PROJECT GOALS

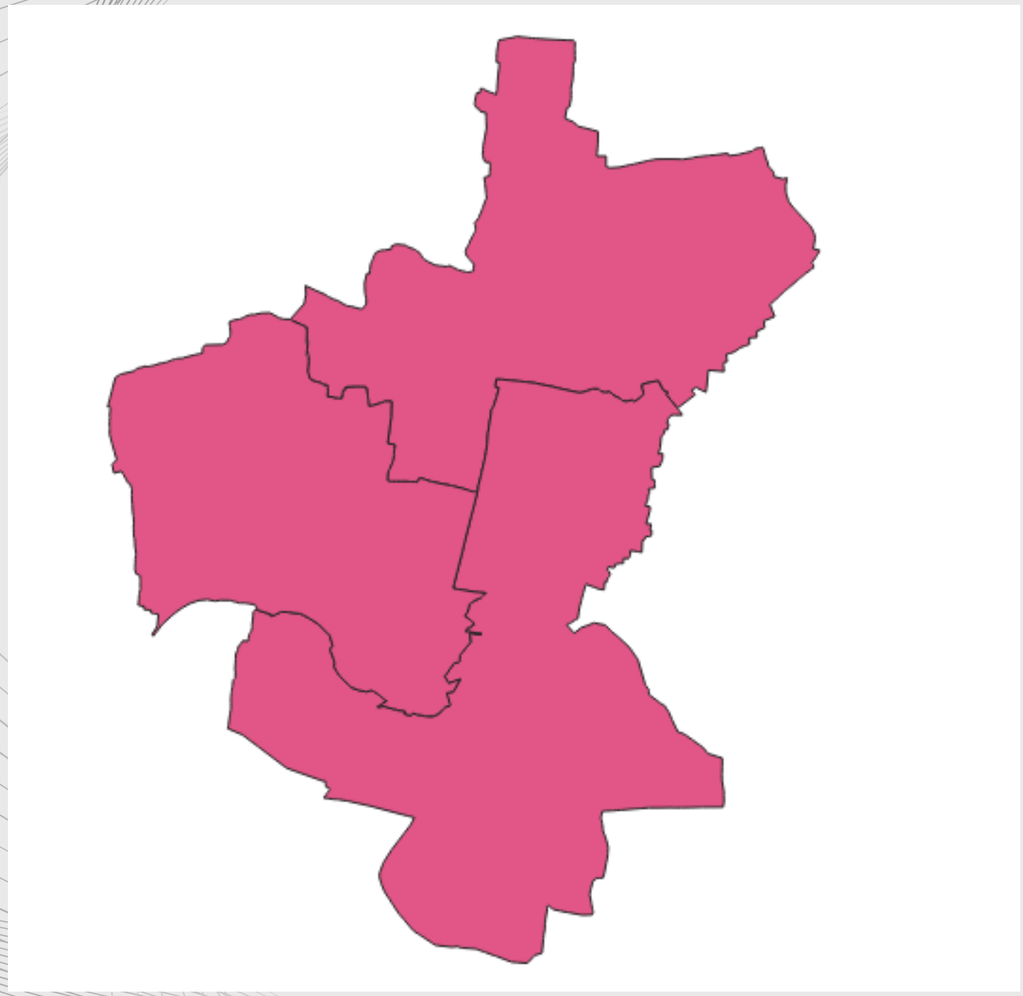
Main Goal

To develop an automated system for road extraction from satellite imagery using Artificial Intelligence and deep learning techniques.

Specific Objectives

- To collect and preprocess high-resolution satellite imagery of the study area in Chennai.
- To prepare training data by digitizing road and non-road features and generating labeled masks for model training.
- To train a deep learning segmentation model using U-Net implemented in PyTorch.
- To automatically detect and classify road pixels and non-road pixels from satellite imagery.
- To evaluate the performance of the model using metrics such as accuracy, F1 Score, precision and recall.
- To generate a pixel level classification visualization using matplotlib

STUDY AREA



Location

The study area is located in Chennai, the capital city of Tamil Nadu, India.

Geographical Coordinates

Latitude: 12.8° N – 13.2° N

Longitude: 80.1° E – 80.3° E

Area Characteristics

- Highly urbanized metropolitan region
- Dense road network including highways, arterial roads, and local streets
- Rapid urban growth and infrastructure development

Purpose of Selecting the Study Area

- Availability of high-resolution satellite imagery
- Presence of diverse road types (major and minor roads)
- Suitable for testing AI-based road extraction from satellite images

Data Used

- Satellite imagery covering the selected ROI
- Digitized road data used as training labels
- Generated road and non-road masks for model training

Data Used

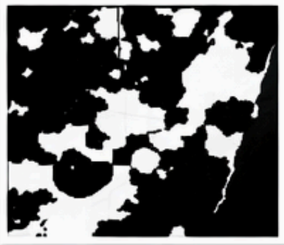
1 Clipped Satellite Imagery

- High-resolution satellite image of Chennai.
- 1 clipped satellite image used for analysis.

2 Digitized Roads and Non-Roads

- **2000** road samples
- **4000** non-road samples

3 Road Mask Dataset



- Binary masks created
- Classes:
  - Road = 1
  - Non-Road = **0**

4 Training Image Patches

- **2565** training patches
- Patch size: **256** × **256** pixels

Dataset Summary

- 1 clipped satellite image
- **2000** digitized road samples
- **4000** non-road samples
- Binary road masks generated
- **2565** training patches (*256 × 256 pixels*)

Tools Used

1. GIS Software

QGIS

- ROI clipping of satellite imagery
- Rasterization & visualization
- Preparation of training data

2. Programming Environment

Python

- Data preprocessing
- Patch generation
- Handling satellite image datasets

3. Deep Learning Framework

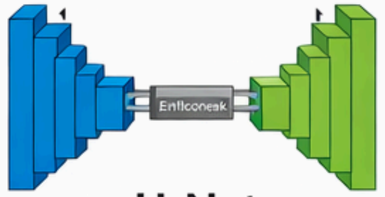
PyTorch

- Model training
- Road segmentation
- Neural network implementation

4. Deep Learning Model

U-Net

- Road extraction model
- Pixel-level segmentation



U-Net

5. Supporting Libraries

NumPy

- Rasterio
- OpenCV

Rasterio

OpenCV

GeoPandas

Methodology

1 Data Import & Pre-processing

- Satellite imagery of the study area was imported and clipped to the ROI using QGIS.
- Image bands were stacked to create a multi-band raster dataset.

2 Training Data Preparation

- Roads and non-road areas were digitized and labeled.
- Two classes were created:
  - Class 1: Road (**2000 samples**)
  - Class 2: Non-road (**4000 samples**)

3 Mask Generation

- The labeled vector data were converted into raster masks to represent road and non-road pixels.

4 Patch Creation

- The raster images and masks were divided into **256 × 256** pixel patches to prepare the dataset for model training.

5 Model Training

- A deep learning segmentation model based on **U-Net** was trained using the prepared image patches and masks.

6 Accuracy Assessment

- Model performance was evaluated using prediction results on validation data.

7 Output Visualization

- Extracted road networks were visualized and analyzed in QGIS. • Accuracy: 99.27% • IoU: 0.67 • F1 Score: 0.80

DIGITIZATION OF ROAD AND NON- ROADS

File Edit Format Run Options Window Help

```
import rasterio
from rasterio.features import shapes
import geopandas as gpd
import numpy as np
import cv2
import pandas as pd
from shapely.geometry import shape
# INPUT FILES
image_path = r"D:/mapjam/output/chennai#_stack_raster.tif"
output_shp = r"D:/mapjam/output/training_samples1.shp"
# READ RASTER
with rasterio.open(image_path) as src:
    img = src.read()
    transform = src.transform
    crs = src.crs
img = np.moveaxis(img,0,-1)
# EXTRACT BANDS (assumes multispectral image)
blue  = img[:,:,0]
green = img[:,:,1]
red   = img[:,:,2]
# if NIR exists
if img.shape[2] >= 4:
    nir = img[:,:,3]
else:
    nir = red
# NDVI (remove vegetation)
ndvi = (nir - red) / (nir + red + 0.0001)
vegetation_mask = ndvi > 0.3
# BRIGHTNESS DETECTION (roads usually bright)
gray = cv2.normalize(red,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
blur = cv2.GaussianBlur(gray,(5,5),0)
_, bright = cv2.threshold(blur,210,255,cv2.THRESH_BINARY)
# remove vegetation
bright[vegetation_mask] = 0
# MORPHOLOGICAL CLEANING
kernel = np.ones((5,5),np.uint8)
roads = cv2.morphologyEx(bright,cv2.MORPH_CLOSE,kernel)
roads = cv2.dilate(roads,kernel,iterations=1)
road_mask = (roads>0).astype(np.uint8)
# NON ROAD MASK
```

```
# NON ROAD MASK
nonroad_mask = np.where(road_mask==0,1,0)
# VECTORIZE ROADS
road_geoms=[]
road_classes=[]
for geom,val in shapes(road_mask,transform=transform):
    if val==1:
        road_geoms.append(shape(geom))
        road_classes.append(1)
roads_gdf = gpd.GeoDataFrame(
    {"class":road_classes},
    geometry=road_geoms,
    crs=crs
)
# PROJECT TO METERS
roads_proj = roads_gdf.to_crs(epsg=32644)
# buffer roads
roads_proj["geometry"]=roads_proj.buffer(4)
# remove very small polygons
roads_proj = roads_proj[roads_proj.geometry.area > 50]
roads_gdf = roads_proj.to_crs(crs)
# VECTORIZE NONROADS
nonroad_geoms=[]
nonroad_classes=[]
for geom,val in shapes(nonroad_mask,transform=transform):
    if val==1:
        nonroad_geoms.append(shape(geom))
        nonroad_classes.append(2)
nonroads_gdf = gpd.GeoDataFrame(
    {"class":nonroad_classes},
    geometry=nonroad_geoms,
    crs=crs
)
```

```
# BALANCE DATASET
# if too many roads, randomly reduce them
max_roads = 2000
if len(roads_gdf) > max_roads:
    roads_gdf = roads_gdf.sample(max_roads, random_state=1)
# generate enough nonroads
target_nonroads = len(roads_gdf) * 2
if len(nonroads_gdf) < target_nonroads:
    additional = nonroads_gdf.sample(target_nonroads, replace=True, random_sta
    nonroads_gdf = additional
# MERGE DATA
training_data = gpd.GeoDataFrame(
    pd.concat([roads_gdf,nonroads_gdf],ignore_index=True),
    crs=crs
)
# SAVE OUTPUT
training_data.to_file(output_shp)
print("Training dataset created")
print("Road samples:",len(roads_gdf))
print("Nonroad samples:",len(nonroads_gdf))
print("Output:",output_shp)
```

ROAD MASK & TRAINING PATCHES CREATION

File Edit Format Run Options Window Help

```
# INPUT FILES
image_path = r"D:\mapjam\output\chennai#_stack_raster.tif"
mask_path = r"D:\mapjam\output\roads_mask.tif"
output_img_dir = r"D:\mapjam\dataset\images"
output_mask_dir = r"D:\mapjam\dataset\masks"
patch_size = 256
os.makedirs(output_img_dir, exist_ok=True)
os.makedirs(output_mask_dir, exist_ok=True)
# READ RASTER
with rasterio.open(image_path) as src:
    img = src.read()
with rasterio.open(mask_path) as src:
    mask = src.read(1)
# convert mask values
mask[mask == 2] = 0
# convert image shape
img = np.moveaxis(img,0,-1)
height, width = mask.shape
count = 0
# CREATE PATCHES
stride = 64
for i in range(0, height - patch_size, stride):
    for j in range(0, width - patch_size, stride):
        img_patch = img[i:i+patch_size, j:j+patch_size]
        mask_patch = mask[i:i+patch_size, j:j+patch_size]
        # count road pixels
        road_pixels = np.sum(mask_patch)
        # keep all road patches
        if road_pixels > 0:
            keep = True
        else:
            # randomly keep some background patches
            keep = np.random.rand() < 0.2
        if not keep:
            continue
        # save patch
        np.save(os.path.join(output_img_dir, f"img_{count}.npy"), img_patch)
        np.save(os.path.join(output_mask_dir, f"mask_{count}.npy"), mask_patch)
        count += 1
print("Total patches created:", count)
```

PROJECT ENVIRONMENT SETUP AND DATASET UPLOAD

```
!pip install numpy torch torchvision matplotlib
```

Show hidden output

```
import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
```

```
from google.colab import files
files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to e

```
Saving dataset.zip to dataset.zip
Buffered data was truncated after reaching the output size limit.
```

```
!unzip dataset.zip
```

```
import os
import numpy as np
import torch
from torch.utils.data import Dataset
class RoadDataset(Dataset):
    def __init__(self, image_dir, mask_dir):
        self.images = sorted(os.listdir(image_dir))
        self.image_dir = image_dir
        self.mask_dir = mask_dir
    def __len__(self):
        return len(self.images)
    def __getitem__(self, idx):
        img_path = os.path.join(self.image_dir, self.images[idx])
        mask_path = os.path.join(
            self.mask_dir,
            self.images[idx].replace("img","mask")
        )
        img = np.load(img_path)
        mask = np.load(mask_path)
        # Normalize image
        img = img / 255.0
        img = torch.tensor(img, dtype=torch.float32).permute(2,0,1)
        mask = torch.tensor(mask, dtype=torch.long)
        return img, mask
```

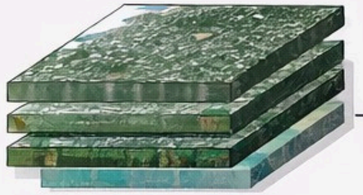
MODEL BUILDING

```
class UNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.down1 = DoubleConv(3,64)
        self.pool1 = nn.MaxPool2d(2)
        self.down2 = DoubleConv(64,128)
        self.pool2 = nn.MaxPool2d(2)
        self.bridge = DoubleConv(128,256)
        self.up1 = nn.ConvTranspose2d(256,128,2,stride=2)
        self.conv1 = DoubleConv(256,128)
        self.up2 = nn.ConvTranspose2d(128,64,2,stride=2)
        self.conv2 = DoubleConv(128,64)
        self.out = nn.Conv2d(64,2,1)
```

```
import torch.nn as nn
class DoubleConv(nn.Module):
    def __init__(self,in_c,out_c):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_c,out_c,3,padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_c,out_c,3,padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True)
        )
    def forward(self,x):
        return self.net(x)
```

```
def forward(self,x):
    d1 = self.down1(x)
    p1 = self.pool1(d1)
    d2 = self.down2(p1)
    p2 = self.pool2(d2)
    bridge = self.bridge(p2)
    u1 = self.up1(bridge)
    u1 = torch.cat([u1,d2],dim=1)
    u1 = self.conv1(u1)
    u2 = self.up2(u1)
    u2 = torch.cat([u2,d1],dim=1)
    u2 = self.conv2(u2)
    return self.out(u2)
```

AI Enabled Road Extraction Model



Satellite Dataset

Data Preprocessing
- Normalization
- Tensor Conversion

Data Preprocessing
- Normalization
- Tensor Conversion

U-Net Model

Encoder

DoubleConv (3 → 64)
MaxPool

DoubleConv (64 → 128)
MaxPool

MaxPool

DoubleConv
(128 → 256)

Decoder

TransposedConv (256 → 128)

DoubleConv (256 → 128)

DoubleConv (256 → 128)

TransposedConv (128 → 64)

DoubleConv (128 → 68)
MaxPool

U-Net Model

Loss Calculation

Weighted Cross Entropy
(w = [1.0 , 5.0]])

Road Segmentation Mask

Predicted Road Mask

Loss Calculation

Loss Function:
Weighted Cross Entropy
(w = [1.0, 5.0])

Evaluation Metrics

- F1 Score
- IoU

DATASET CLEANING AND TRAINING OF MODEL

```python
import os

image_dir = "/content/dataset/images"
mask_dir = "/content/dataset/masks"

images = sorted(os.listdir(image_dir))
masks = set(os.listdir(mask_dir))

for img in images:

    mask_name = img.replace("img", "mask")

    if mask_name not in masks:
        os.remove(os.path.join(image_dir, img))

print("Dataset cleaned")
```

```
Dataset cleaned
```

```python
epochs = 10
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for imgs, masks in loader:
        imgs = imgs.to(device)
        masks = masks.to(device)
        preds = model(imgs)
        loss = loss_fn(preds, masks)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    avg_loss = total_loss / len(loader)
    print(f"Epoch {epoch+1}/{epochs} Loss:{avg_loss:.4f}")
```

```
Epoch 1/10 Loss:0.0186
Epoch 2/10 Loss:0.0190
Epoch 3/10 Loss:0.0202
Epoch 4/10 Loss:0.0180
Epoch 5/10 Loss:0.0177
Epoch 6/10 Loss:0.0196
Epoch 7/10 Loss:0.0187
Epoch 8/10 Loss:0.0169
Epoch 9/10 Loss:0.0174
Epoch 10/10 Loss:0.0168
```

ACCURACY ASSESSMENT REPORT

```
import torch
model.eval()
TP = 0
TN = 0
FP = 0
FN = 0
with torch.no_grad():
    for imgs, masks in loader:
        imgs = imgs.to(device)
        masks = masks.to(device)
        outputs = model(imgs)
        preds = torch.argmax(outputs, dim=1)
        TP += ((preds == 1) & (masks == 1)).sum().item()
        TN += ((preds == 0) & (masks == 0)).sum().item()
        FP += ((preds == 1) & (masks == 0)).sum().item()
        FN += ((preds == 0) & (masks == 1)).sum().item()
# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP + 1e-6)
recall = TP / (TP + FN + 1e-6)
f1_score = 2 * precision * recall / (precision + recall + 1e-6)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1_score)
```

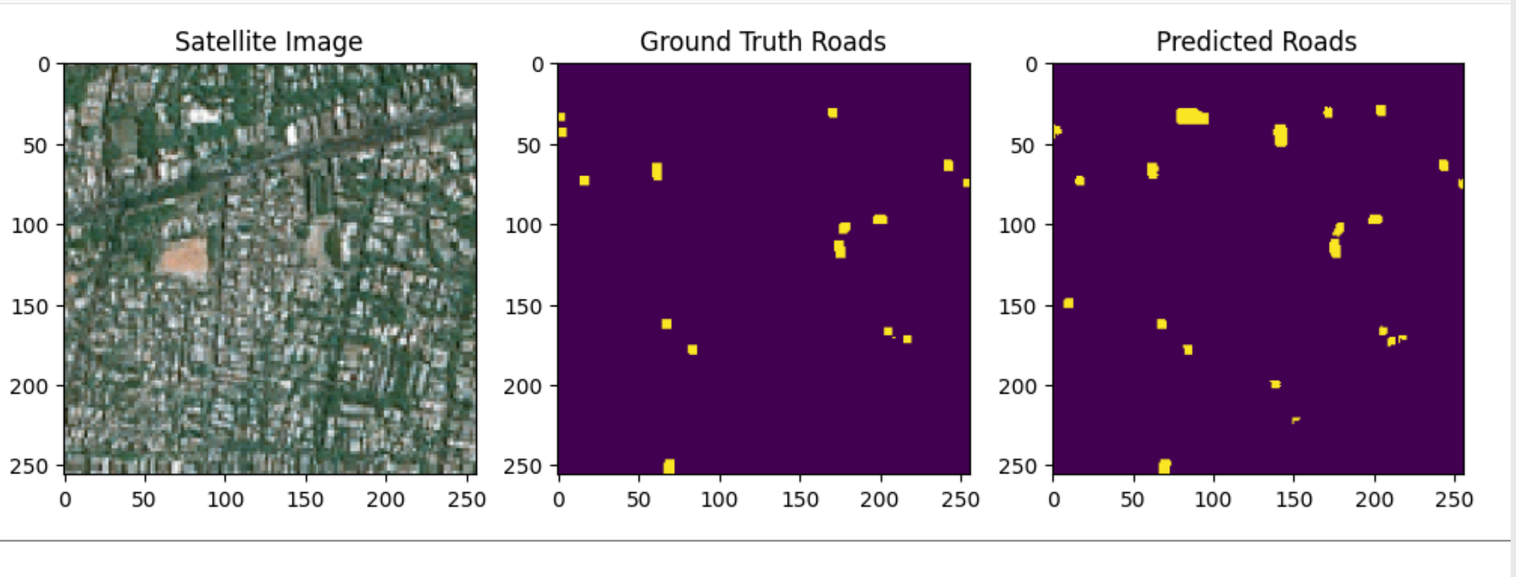
	precision	recall	f1-score
1	0.99	0.99	0.99
2	0.98	0.98	0.98
accuracy			0.99
macro avg	0.98	0.98	0.98
weighted avg	0.99	0.99	0.99

Precision (0.99): Out of all pixels the model identified as "Road," 99% of them were actually roads. This means there is almost zero "false alarm".

Recall (0.99): Out of all the actual roads in Chennai imagery, the model successfully found 99% of them.

F1-Score (0.99): This is the harmonic mean of Precision and Recall. A score of 0.99 indicates an extremely robust model that balances accuracy and reliability perfectly.

VISUALIZATION




```
import matplotlib.pyplot as plt
model.eval()
img, mask = dataset[5]
with torch.no_grad():
    pred = model(img.unsqueeze(0).to(device))
    pred = torch.argmax(pred, dim=1).cpu().numpy()[0]
plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.title("Satellite Image")
plt.imshow(img.permute(1,2,0))
plt.subplot(1,3,2)
plt.title("Ground Truth Roads")
plt.imshow(mask)
plt.subplot(1,3,3)
plt.title("Predicted Roads")
plt.imshow(pred)
plt.show()
```

The predicted road map closely resembles the ground truth mask, showing that the model successfully learned road features.

Some minor differences or missing segments may occur due to limited training data, complex urban patterns, similar textures between roads and surrounding area, .

CONCLUSION & KEY INSIGHTS

Conclusion

- A deep learning–based road extraction framework was successfully developed using the U-Net segmentation model.
- High-resolution satellite imagery of the study area was processed and converted into training patches.
- The model was trained to classify road and non-road pixels using manually generated ground truth masks.
- The trained model demonstrated strong performance in detecting road networks from satellite imagery.

Key Insights

- The model achieved high accuracy (~99%), showing reliable classification between road and non-road regions.
- Proper dataset preparation, patch generation, and balanced training samples significantly improved model learning.
- The approach demonstrates that deep learning can automate road extraction from satellite imagery, reducing manual digitization effort.

THANK YOU